



前言（可跳过）

hello，大家好我是 Tian-Feng，今天介绍一些stable diffusion⁺的原理，内容通俗易懂，因为我平时也玩Ai绘画嘛，所以就像写一篇文章说明它的原理，这篇文章写了真滴挺久的，如果对你有帮助的话，希望点个赞，谢谢。

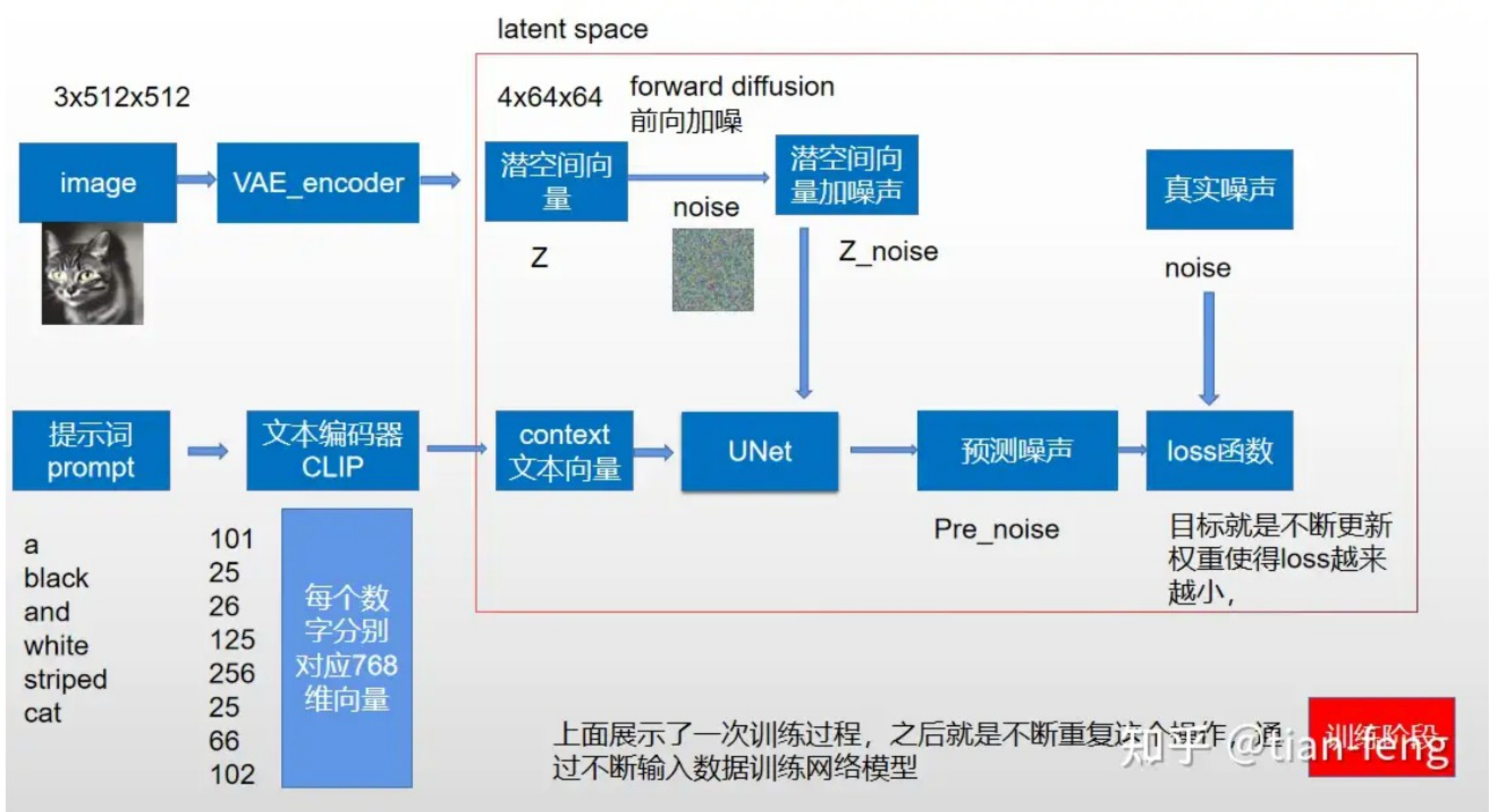
stable diffusion作为Stability-AI开源图像生成模型，其出现也是不逊于ChatGPT，其发展势头丝毫不差于midjourney⁺，加上其众多插件的加持，其上线也是无线拔高，当然，手法上也稍微比midjourney复杂点。[论文](#) [源码](#)

至于为什么开源，创始人：我这么做的原因是，我认为这是共同叙事（shared narrative）的一部分，有人需要公开展示发生了什么。再次强调，这应该默认就是开源的。因为价值不存在于任何专有模型或数据中，我们将构建可审计（auditable）的开源模型，即使其中包含有许可数据。话不多说，开整。

二、stable diffusion

对于上面原论文的图片可能小伙伴理解有困难，但是不打紧，我会把上面图片分成一个个单独的模块进行解读，最后组合在一起，相信你们一定可以理解图片每一步干了什么事。

首先，我会画一个简化模型图对标原图，以方便理解。让我们从训练阶段开始，可能你们发现少了VAEdecoder，这是因为我们训练过程是在潜空间完成，decoder⁺我们放在第二阶段采样阶段说，我们所使用的stablediffusion webui画图通常是在采样阶段，至于训练阶段，目前我们大多数普通人是根本完成不了的，它所需要训练时间应该可以用GPUyear来计量，（单V100的GPU要一年时间），如果你有100张卡，应该可以一个月完成。至于ChatGPT光电费上千万美金，上万GPU集群，感觉现在AI拼的就是算力。又扯远了，come back



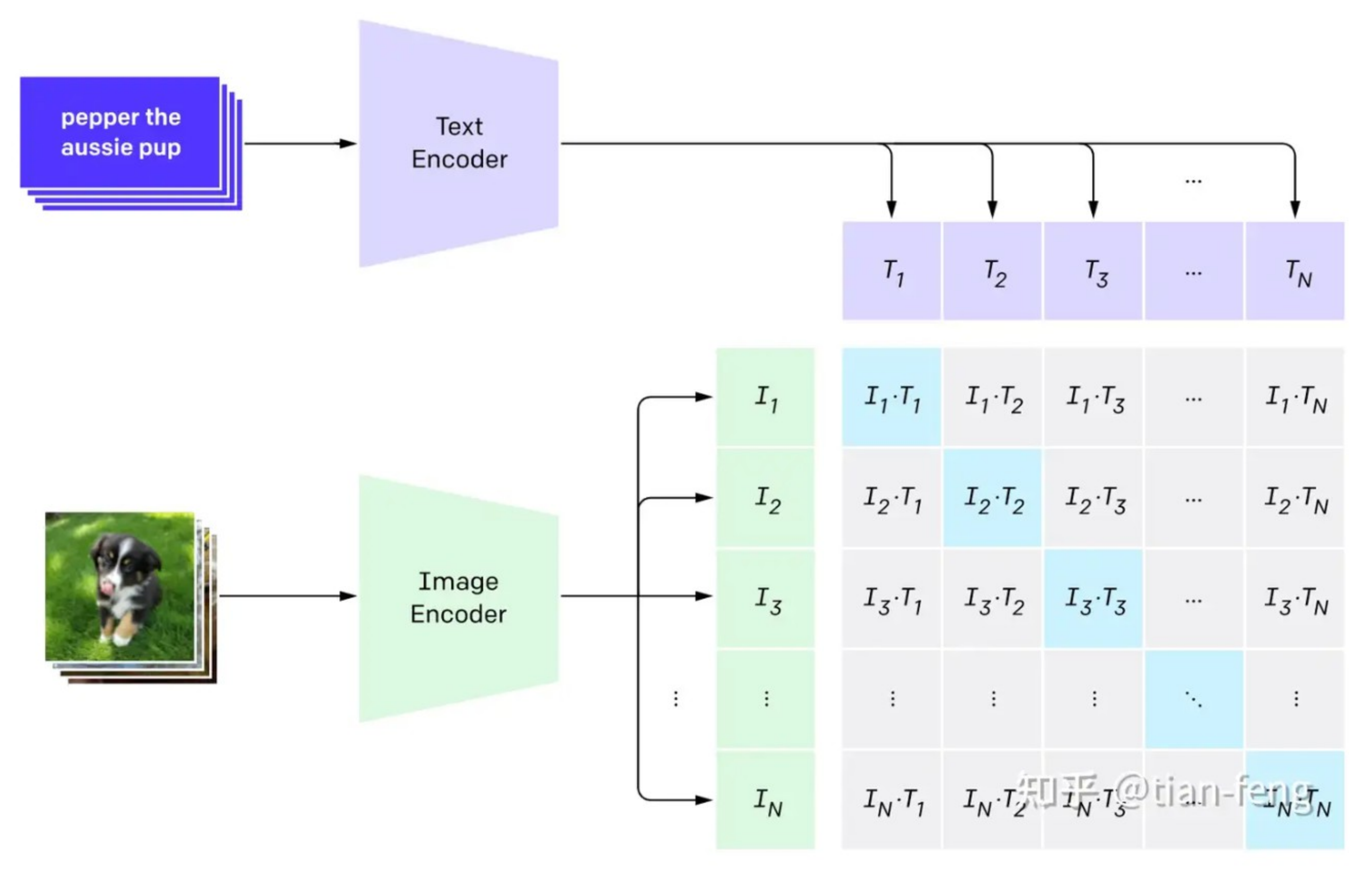
1.clip

我们先从提示词开始吧，我们输入一段提示词a black and white striped cat（一条黑白条纹的猫），clip会把文本对应一个词表，每个单词标点符号都有相对应的一个数字，我们把每个单词叫做一个token，之前stablediffusion输入有限制只能75个单词（现在没了），也就是75个token，看上面你可能发现6个单词怎么对应8个token，这是因为还包含了起始token和结束token，每个数字又对应这一个768维的向量，你可以看作每个单词的身份证，而且意思非常相近的单词对应的768维向量也基本一致。经过clip我们得到了一个（8,768）的对应图像的**文本向量**。

stable diffusion所使用的是openAi的clip的预训练模型，就是别人训练好的拿来用就行，那clip是怎么训练出来的呢？他是怎么把图片和文字信息对应呢？（下面扩展可看可跳过，不影响理解，只需要知道它是用来把提示词转成对应生成图像的文本向量即可）

CLIP需要的数据为图像及其标题，数据集中大约包含4亿张图像及描述。应该是直接爬虫得来，图像信息直接作为标签，训练过程如下：

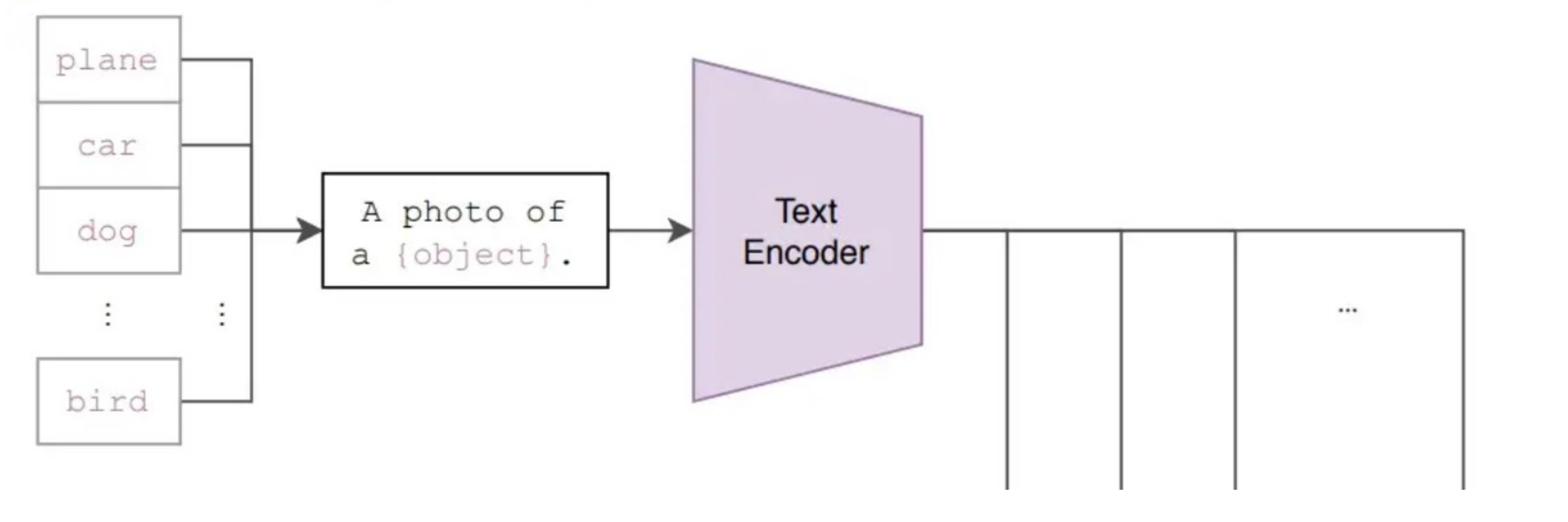
CLIP 是图像编码器和文本编码器的组合，使用两个编码器对数据分别进行编码。然后使用**余弦距离**比较结果嵌入，刚开始训练时，即使文本描述与图像是相匹配的，它们之间的相似性肯定也是很低的。



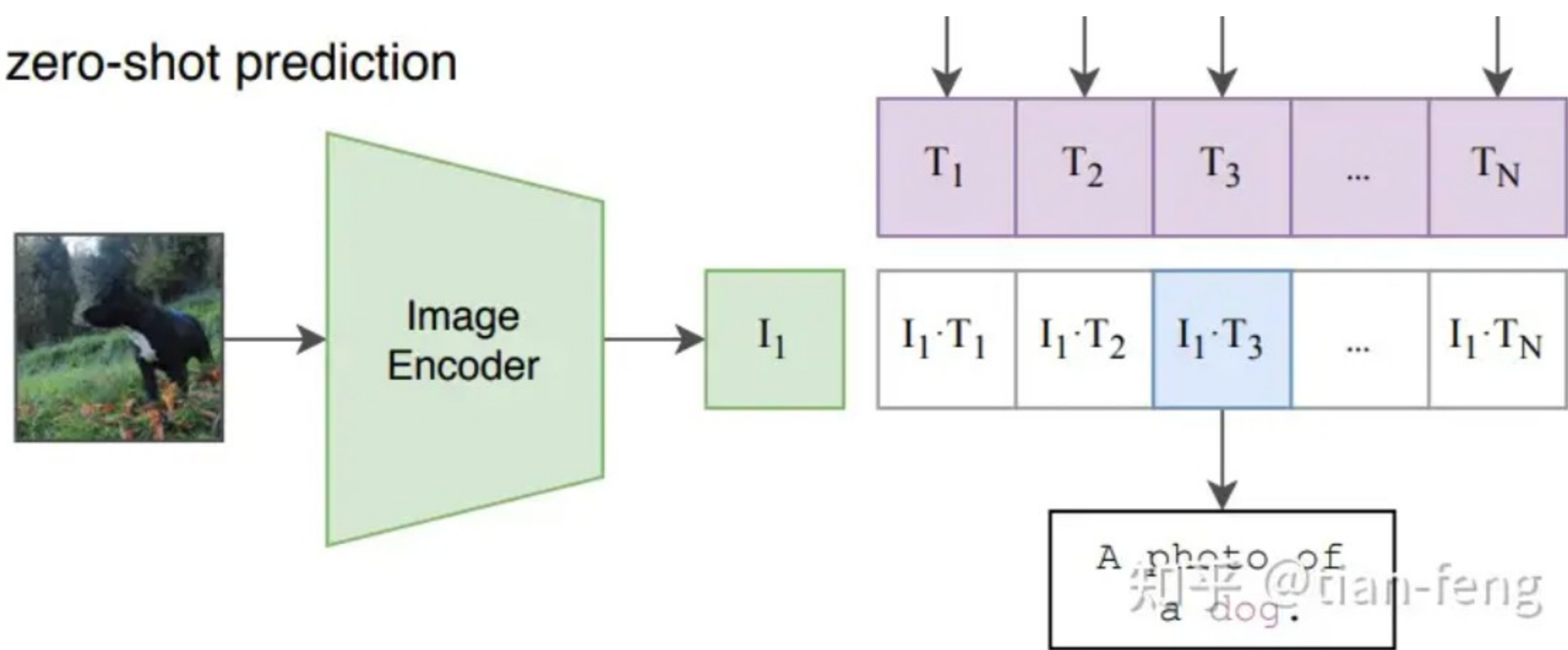
随着模型的不断更新，在后续阶段，编码器对图像和文本编码得到的嵌入会逐渐相似。在整个数据集中重复该过程，并使用大batch size的编码器，最终能够生成一个嵌入向量，其中狗的图片 and 句子「一条狗的图片」之间是相似的。

给一些提示文本，然后每种提示算相似度，找到概率最高的即可

(2) Create dataset classifier from label text

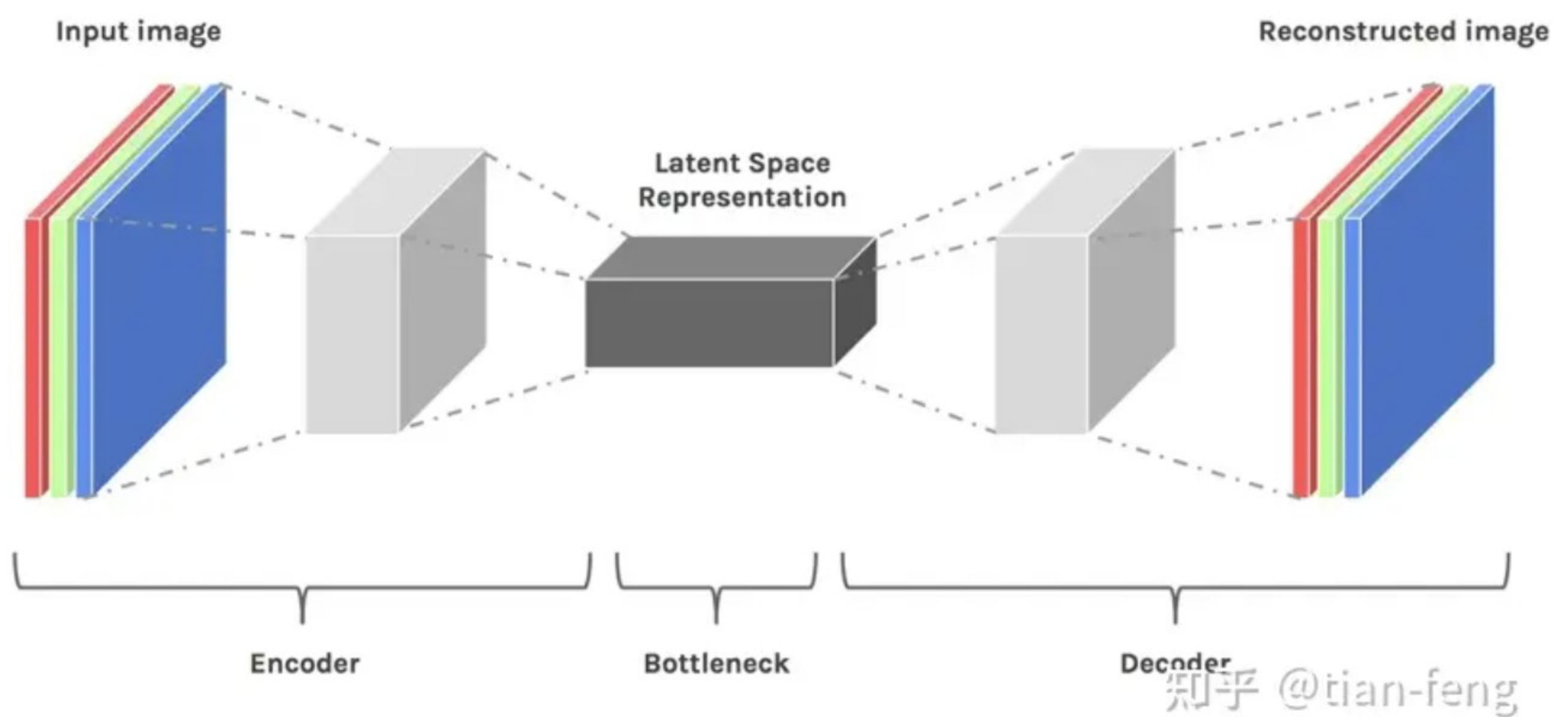


(3) Use for zero-shot prediction



2.diffusion model

上面我们已经得到了unet的一个输入了，我们现在还需要一个噪声图像的输入，假如我们输入的是一张3x512x512的猫咪图像，我们不是直接对猫咪图像进行处理，而是经过VAE encoder把512x512图像从pixel space（像素空间）压缩至latent space（潜空间）4x64x64进行处理，数据量小了接近64倍。

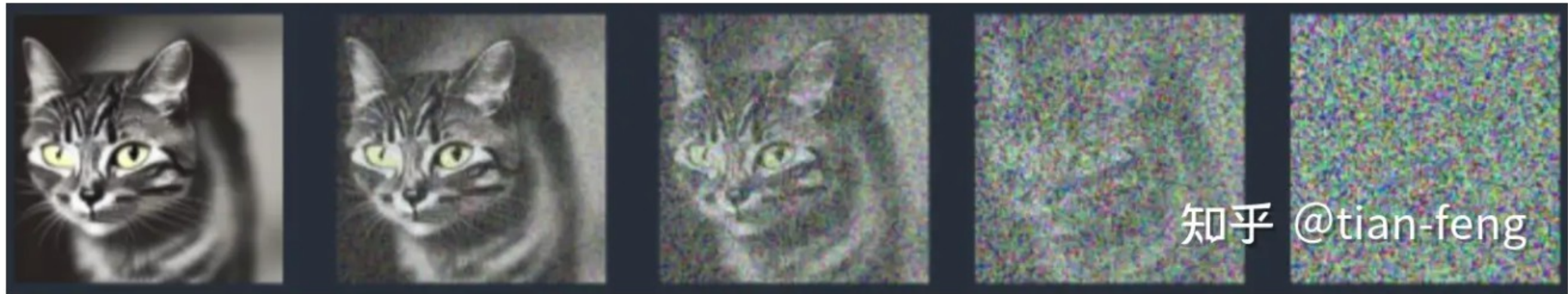


潜在空间简单的说是对压缩数据的表示。所谓压缩指的是用比原始表示更小的数位来编码信息的过程。维度降低会丢失一部分信息，然而在某些情况下，降维不是件坏事。通过降维我们可以过滤掉一些不太重要的信息你，只保留最重要的信息。

得到潜空间向量后，现在来到扩散模型⁺，为什么图像加噪后能够还原，秘密都在公式里，这里我以DDPM论文作为理论讲解，[论文](#)，当然还有改进版本DDIM等等，感兴趣自己看

forward diffusion⁺（前向扩散）

- 首先是forward diffusion（前向扩散），也就是加噪过程，最后就快变成了个纯噪声
- 每一个时刻都要添加高斯噪声⁺，后一时刻都是由前一刻是增加噪声得到



那么是否我们每一次加噪声都要从前一步得到呢，我们能不能想要第几步加噪图像就能得到呢？答案是YES，作用是：我们训练过程中对图像加噪是随机的，假如 我们随机到100步噪声，（假设设置时间步数200步），如果要从第一步加噪，得到第二步，循环往复，太费时间了，其实这些加的噪声有规律的，我们现在的目标就是只要有原始图像X0，就可以得到任意时刻图像加噪声的图像，而不必一步一步得到想要的噪声图像。

$\alpha_t = 1 - \beta_t$ β 要越来越大，论文中0.0001到0.002，从而 α 也就是要越来越小
t0时刻, $\alpha_0=1, \beta_t=0$
★ $x_t = \sqrt{a_t}x_{t-1} + \sqrt{1 - \alpha_t}z_1$ 一开始加点噪就有效果，越往后得加噪越多才行
 x_t 表示t时刻图像, z 表示噪声
但是现在咱们只能知道后一时刻分布是由前一时刻加噪得到的，但是整个序列咋算？
如果一个个来计算，那也太慢了吧，能不能直接 x_t 由 x_0 直接就能算出来呢，咱们来试试看：

$$x_{t-1} = \sqrt{a_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}z_2$$

一步步来，再往前一时刻咱们也能算

$$x_t = \sqrt{a_t}(\sqrt{a_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}z_2) + \sqrt{1 - \alpha_t}z_1$$

其中每次加入的噪声都服从高斯分布 $z_1, z_2, \dots \sim \mathcal{N}(0, \mathbf{I})$

$$= \sqrt{a_t a_{t-1}}x_{t-2} + (\sqrt{a_t(1 - \alpha_{t-1})}z_2 + \sqrt{1 - \alpha_t}z_1)$$

这步就展开而已

$$= \sqrt{a_t a_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}}z_2'$$

$$\mathcal{N}(0, \sigma_1^2 \mathbf{I}) + \mathcal{N}(0, \sigma_2^2 \mathbf{I}) \sim \mathcal{N}(0, (\sigma_1^2 + \sigma_2^2) \mathbf{I})$$

$$= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}z_t$$

不断往里套，就能发现规律了，其实就是累乘

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

$$X \sim N(\mu, \sigma^2)$$

$$\mathcal{N}(0, 1 - \alpha_t)$$

$$\mathcal{N}(0, a_t(1 - \alpha_{t-1}))$$

$$\mathcal{N}(0, \sigma_1^2 \mathbf{I}) + \mathcal{N}(0, \sigma_2^2 \mathbf{I}) \sim \mathcal{N}(0, (\sigma_1^2 + \sigma_2^2) \mathbf{I})$$

$$= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}z_t$$

不断往里套，就能发现规律了，其实就是累乘

$$x_t = \sqrt{a_t a_{t-1} a_{t-2}}x_{t-3} + \sqrt{1 - \alpha_t \alpha_{t-1} \alpha_{t-2}}z_3$$

发现规律了没有，就是 α 的累乘

高斯分布公式

带入到 $\star$ 中可以得到下式

之后把 x_{t-2} 带入，直到 x_1 ，这里就是相加后仍服从高斯做了化简

就有了 x_t 和 x_0 的关系

这个公式告诉我们，任意时刻的分布都可以通过 x_0 初始状态算出来，一步到位

这也就是咱们第一个最核心的公式，如何进行Diffusion，代码中一会要用

我来对上述作讲解，其实该标住的我都标的很清楚了，

第一， α_t 范围0.9999-0.998，

第二，图像加噪是符合高斯分布的，也就是在潜空间向量加的噪声是符合均值为0，方差为1的，将 x_{t-1} 带入 x_t 中，为什么两项可以合并，因为 $z_1 z_2$ 都是符合高斯分布，那么他们相加 z_2' 也符合，并且它们的方差和为新的方差，所有把他们各自的方差求和，（那个带根号的是标准差），如果你无法理解，可以把它当做一个定理。在多说一句，对 $Z \rightarrow a+bZ$,那么 Z 的高斯分别也从 $(0, \sigma) \rightarrow (a, b\sigma)$ ，现在我们得到了 x_t 跟 x_{t-2} 的关系

第三，如果你再把 x_{t-2} 带入，得到与 x_{t-3} 的关系，并且找到规律，就是 α 的累乘，最后得到 x_t 与 x_0 的关系式，现在我们可以根据这个式子直接得到任意时刻的噪声图像。

第四，因为图像初始化噪声是随机的，假设你设置的时间步数（timesteps）为200，就是把0.9999-0.998区间等分为200份，代表每个时刻的 α 值，根据 x_t 和 x_0 的公式，因为 α 累乘（越小），可以看出越往后，噪声加的越快，大概1-0.13的区间，0时刻为1，这时 x_t 代表图像本身，200时刻代表图像大概 α 为0.13噪音占据了0.87，因为是累乘所以噪声越加越大，并不是一个平均的过程。

Captured by FireShot Pro: 31 十二月 2024, 09:20:45

https://getfireshot.com

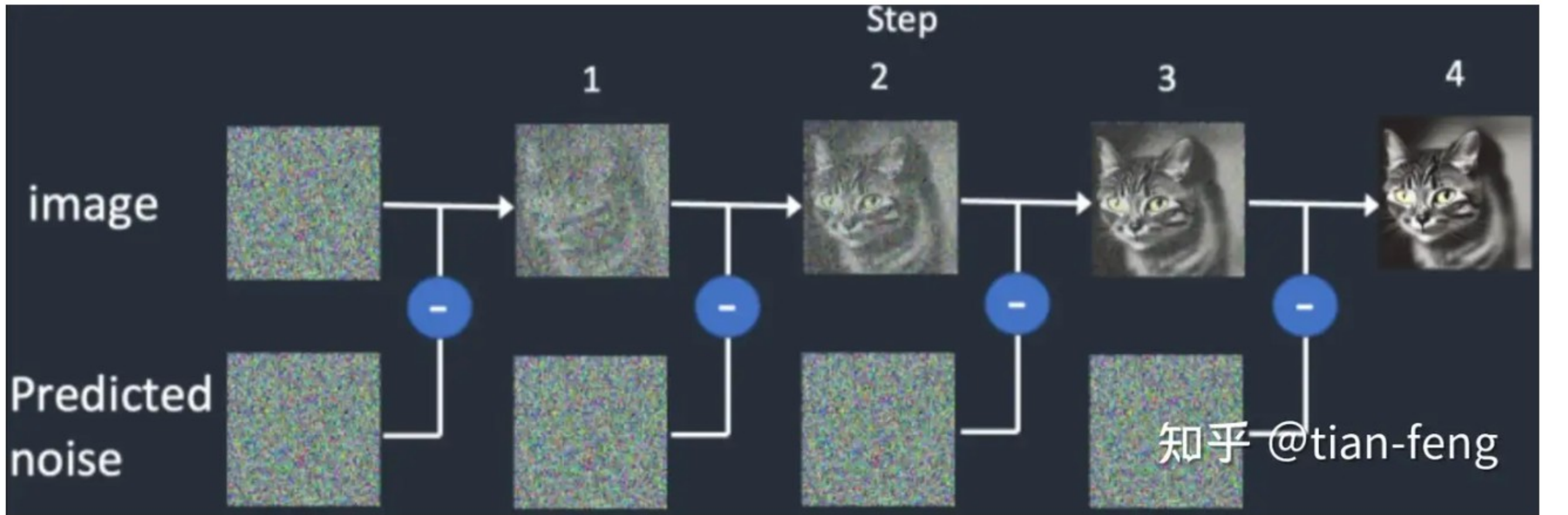
第五，补充一句，重参数化技巧(Reparameterization Trick)

如果 $X(u,\sigma^2)$ ，那么 X 可以写成 $X=\mu+\sigma Z$ 的形式，其中 $Z\sim N(0,1)$ 。这就是重参数化技巧。

重参数化技巧，就是从一个分布中进行采样，而该分布是带有参数的，如果直接进行采样（采样动作是离散的，其不可微），是没有梯度信息的，那么在BP反向传播的时候就不会对参数梯度进行更新。重参数化技巧可以保证我们从进行采样，同时又能保留梯度信息。

逆向扩散（reverse diffusion）

- 前向扩散完毕，接下来是逆向扩散（reverse diffusion），这个可能比上面那个难点，如何根据一个噪声图像一步步得到原图呢，这才是关键，



- 逆向开始，我们目标是 X_t 噪声图像得到无噪声的 X_0 ，先从 X_t 求 X_{t-1} 开始，这里我们先假设 X_0 是已知（先忽略为什么假设已知），后面会替换它，至于怎么替换，前向扩散不是已知 X_t 和 X_0 的关系吗，现在我们已知的是 X_t ,反过来用 X_t 来表示 X_0 ，但是还有一个 Z 噪声是未知的，这个时候就要Unet上场了，它需要把噪声预测出来。
- 这里借助贝叶斯公式[★]（就是条件概率），我们借助贝叶斯公式结果，[以前写过一个文档](#)

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$
$$P(A \cap B) = P(A \mid B)P(B)$$

$$P(A \cap B) = P(B \mid A)P(A)$$
$$P(A \mid B)P(B) = P(B \mid A)P(A)$$
$$P(A \mid B) = \frac{P(B|A)P(A)}{P(B)}$$

知乎 @tian-feng

就是已知Xt求Xt-1，反向我们不知道怎么求，但是求正向，如果我们已知X0那么这几项我们都可以求出来。

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$$

↑

逆向过程要求解的

↑

不会求，来个贝叶斯公式

这三项不就是咱们刚才求的嘛，这是已知条件了，可以直接展开

$$q(\mathbf{x}_{t-1}|\mathbf{x}_0) \quad \sqrt{\bar{a}_{t-1}} \mathbf{x}_0 + \sqrt{1 - \bar{a}_{t-1}} \mathbf{z} \quad \sim \mathcal{N}(\sqrt{\bar{a}_{t-1}} \mathbf{x}_0, 1 - \bar{a}_{t-1})$$
$$q(\mathbf{x}_t|\mathbf{x}_0) \quad \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \mathbf{z} \quad \sim \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, 1 - \bar{\alpha}_t)$$
$$q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) = \sqrt{\bar{a}_t} \mathbf{x}_{t-1} + \sqrt{1 - \bar{\alpha}_t} \mathbf{z} \quad \sim \mathcal{N}(\sqrt{\bar{a}_t} \mathbf{x}_{t-1}, 1 - \bar{\alpha}_t)$$

这三项都通过前向过程能算出来，分布也列出来了

代入高斯分布

$$\propto \exp\left(-\frac{1}{2}\left(\frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0)^2}{1 - \bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0)^2}{1 - \bar{\alpha}_t}\right)\right)$$

把标准正太分布展开后，乘法就相当于加，除法就相当于减，把他们汇总

知乎 @tian-feng

开始解读，既然这三项都符合高斯分别，那带入高斯分布（也可以叫正态分布），它们相乘为什么等于相加呢，因为e2 * e3 =e2+3，这个能理解吧（属于exp，就是e次方），好，现在我们得到了一个整体式子，接下来继续化简

首先我们把平方展开，未知数现在只有Xt-1，配成AX2+BX+C格式，不要忘了，即使相加也是符合高斯分布，现在我们把原高斯分别公式配成一样的格式，红色就是方差的倒数，把蓝色乘方差除2就得到了均值μ（下面显示是化简的结果，如果你有兴趣自己，自己化简），回归X0，之前说X0假设已知，现在转成Xt（已知）表示，代入μ，现在未知数只剩下Zt，

Captured by FireShot Pro: 31 十二月 2024, 09:20:45

https://getfireshot.com

$$\begin{aligned} &\propto \exp\left(-\frac{1}{2}\left(\frac{(\mathbf{x}_t - \sqrt{\alpha_t}\mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0)^2}{1 - \bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0)^2}{1 - \bar{\alpha}_t}\right)\right) \\ &= \exp\left(-\frac{1}{2}\left(\frac{\mathbf{x}_t^2 - 2\sqrt{\alpha_t}\mathbf{x}_t\mathbf{x}_{t-1} + \alpha_t\mathbf{x}_{t-1}^2}{\beta_t} + \frac{\mathbf{x}_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0\mathbf{x}_{t-1} + \bar{\alpha}_{t-1}\mathbf{x}_0^2}{1 - \bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0)^2}{1 - \bar{\alpha}_t}\right)\right) \\ &= \exp\left(-\frac{1}{2}\left(\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}\right)\mathbf{x}_{t-1}^2 - \left(\frac{2\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{2\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}}\mathbf{x}_0\right)\mathbf{x}_{t-1} + C(\mathbf{x}_t, \mathbf{x}_0)\right)\right) \end{aligned}$$

C那个是常数项，不影响

这个任务中，咱们核心就是求跟 $\mathbf{x}_{t-1}$ 有关的，其他的都不关心现在，这一顿展开就是在给它配方呢

$$\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) = \exp\left(-\frac{1}{2}\left(\frac{1}{\sigma^2}x^2 - \frac{2\mu}{\sigma^2}x + \frac{\mu^2}{\sigma^2}\right)\right)$$

这样就能得到均值和方差了

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\mathbf{x}_0$$

配完化简后的结果，但是我哪有 $\mathbf{x}_0$ 啊现在

之前咱们说 $\mathbf{x}_t$ 可以由 $\mathbf{x}_0$ 计算得到，现在逆一下

$$\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\mathbf{z}_t)$$

$$\tilde{\mu}_t = \frac{1}{\sqrt{a_t}}\left(x_t - \frac{\beta_t}{\sqrt{1 - a_t}}z_t\right)$$

最终结果

但是好像有一个问题， $\mathbf{z}_t$ 是啥，这东西我咋能知道呢？

知乎 @tian-feng

- $\mathbf{z}_t$ 其实就是我们要估计的每个时刻的噪声
- 这里我们使用Unet模型预测
- 模型的输入参数有三个，分别是当前时刻的分布 $\mathbf{x}_t$ 和时刻 t ，还有之前的文本向量，然后输出预测的噪声，这就是整个过程了，

Algorithm 1 Training	Algorithm 2 Sampling
1: repeat 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 3: $t \sim \text{Uniform}(\{1, \dots, T\})$ 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 5: Take gradient descent step on $\nabla_{\theta} \left\ \epsilon - \left[\epsilon_{\theta} \left(\sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t \right) \right] \right\ ^2$ 6: until converged Unet	1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 2: for $t = T, \dots, 1$ do 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$ 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 5: end for 6: return $\mathbf{x}_0$

知乎 @tian-feng

- 上面的Algorithm 1是训练过程，

其中第二步表示取数据，一般来说都是一类猫，狗什么的，或者一类风格的图片，不能乱七八糟什么图片都来，那模型学不了。

第三步是说每个图片随机赋予一个时刻的噪声（上面说过），

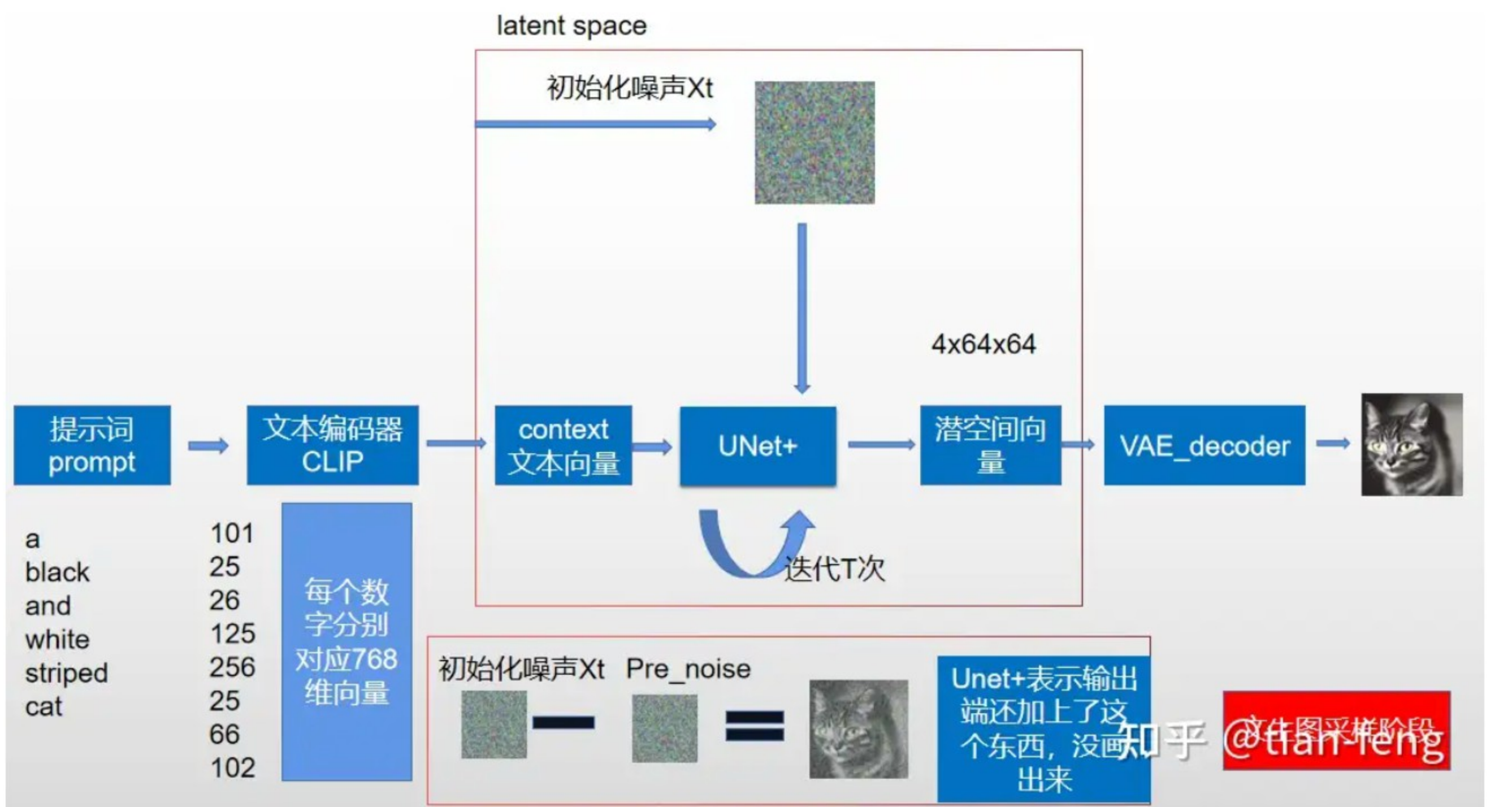
第四步，噪声符合高斯分布，

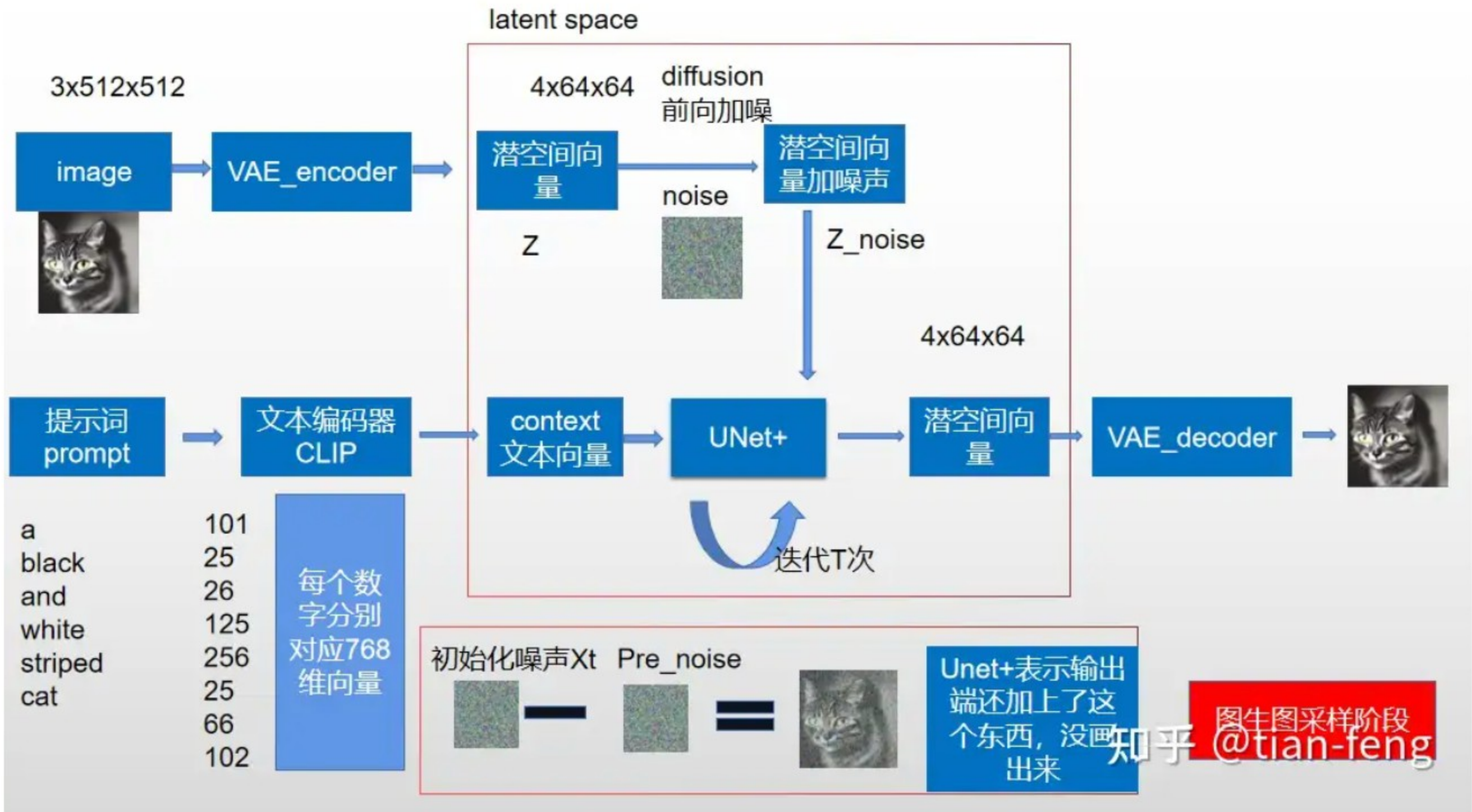
第五步，真实的噪声和预测的噪声算损失（DDPM输入没有文本向量，所有没有写，你就理解为多加了一个输入），更新参数。直到训练的输出的噪声和真实噪声相差很小，Unet模型训练完毕

- 下面我们来到Algorithm2采样过程
1. 不就是说 X_t 符合高斯分布嘛
 2. 执行T次，依次求 X_{t-1} 到 X_0 ，不是T个时刻嘛
 3. X_{t-1} 不就是我们逆向扩散推出的公式， $X_{t-1}=\mu+\sigma Z$ ，均值和方差都是已知的，唯一的未知噪声Z被Unet模型预测出来， ϵ_{θ} 这个是指已经训练好的Unet，

采样图

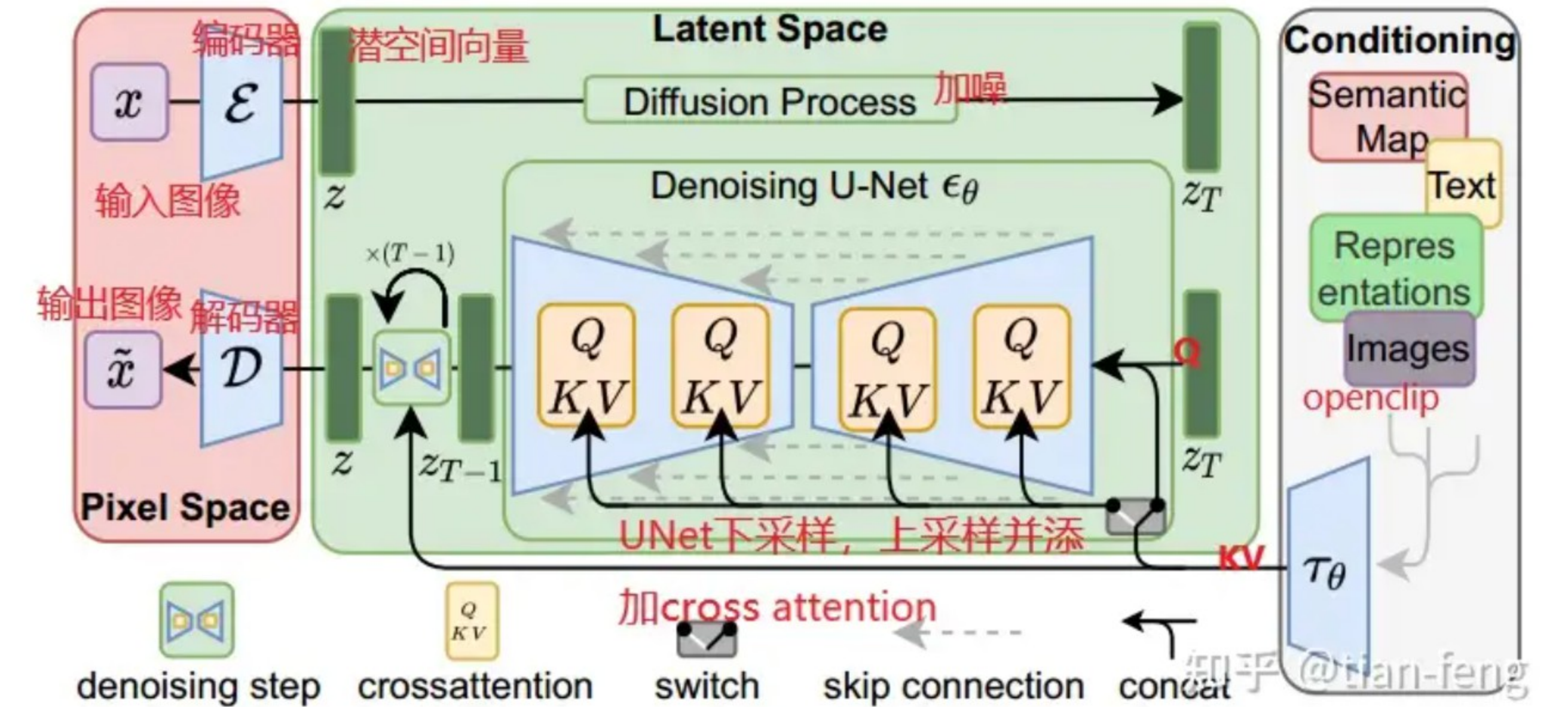
- 为了方便理解，我分别画出文生图和图生图，如果使用stable diffusion webui画图的人一定觉得很熟悉，如果是文生图就是直接初始化一个噪声，进行采样，
- 图生图则是在你原有的基础上加噪声，噪声权重自己控制，webui界面是不是有个重绘幅度，就是这个，
- 迭代次数就是我们webui界面的采样步数，
- 随机种子seed就是我们初始随机得到的一个噪声图，所以如果想要复刻得到一样的图，seed要保持一致





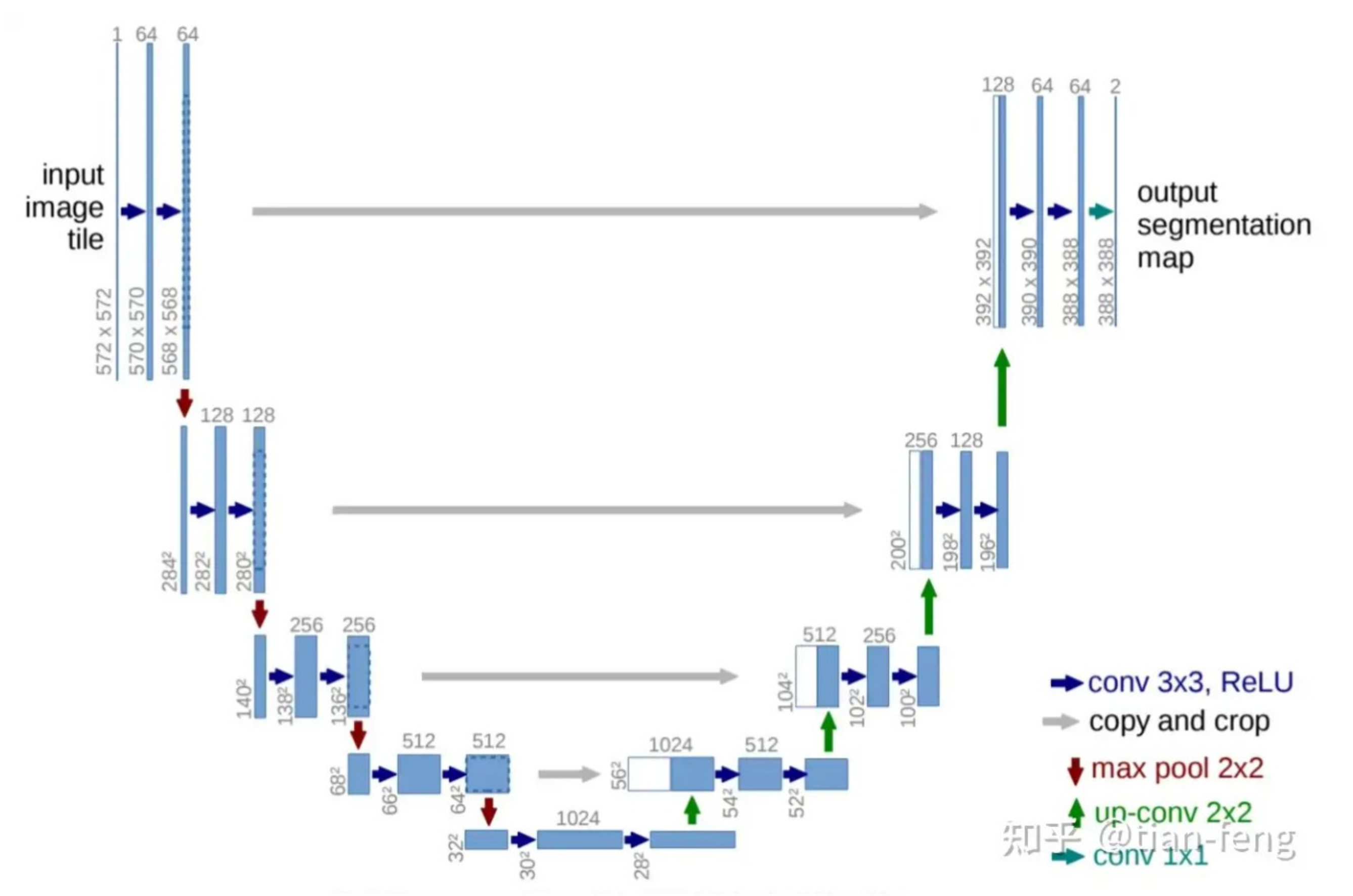
阶段小结

我们现在再来看这张图，除了Unet我没讲（下面会单独介绍），是不是简单多了，最左边不就是像素空间的编码器解码器，最右边就是clip把文本变成文本向量，中间上面的就是加噪，下面就是Unet预测噪声，然后不停的采样解码得到输出图像。这是原论文采样图，没画训练过程。

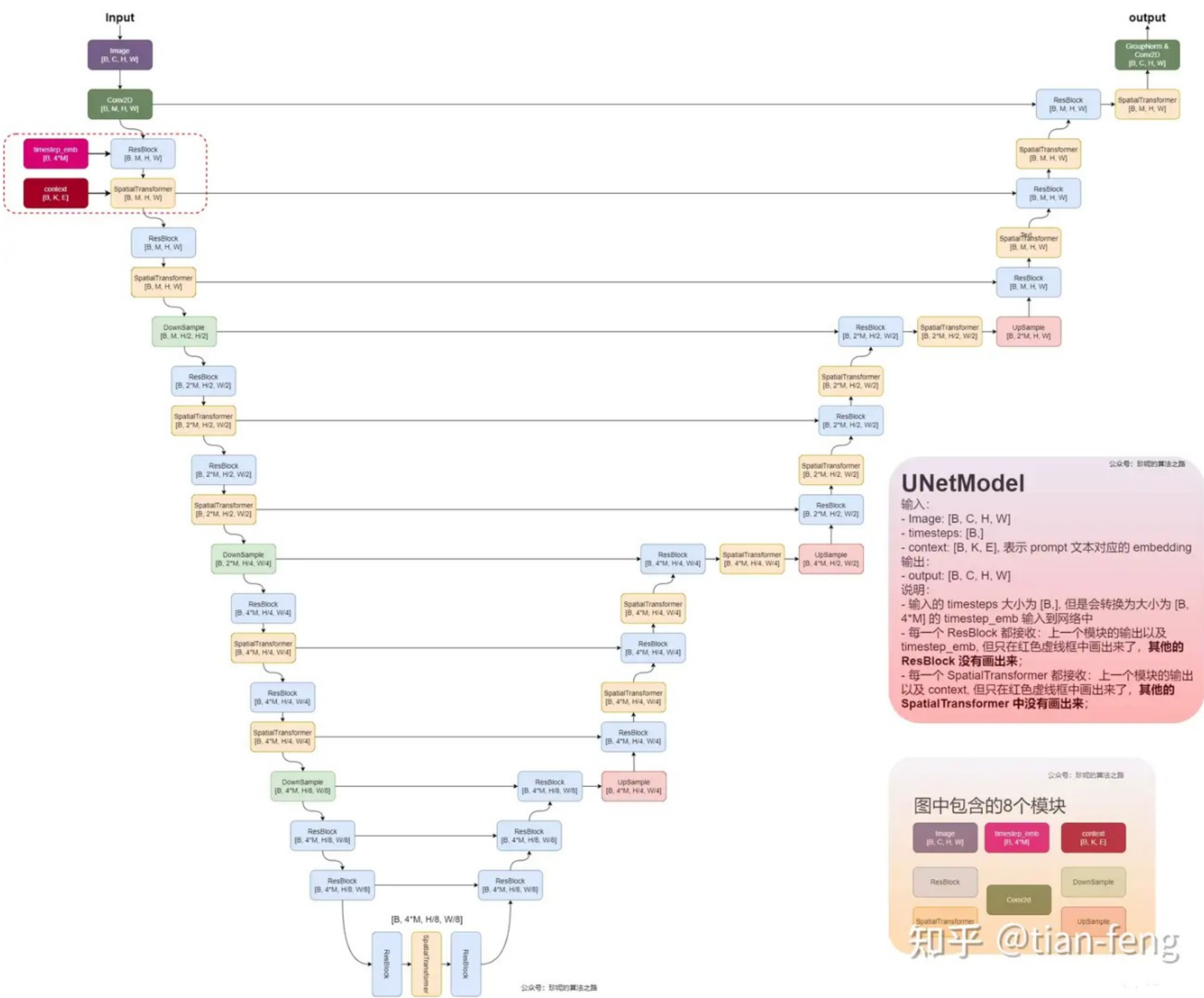


3.Unet model

unet模型相信小伙伴们都或多或少知道，就是多尺度特征融合，像FPN图像金字塔+，PAN，很多都是差不多的思想，一般使用resnet作为backbone（下采样），充当编码器，这样我们就得到多个尺度的特征图，然后在上采样过程中，上采样拼接（之前下采样得到的特征图），上采样拼接，这是一个普通的Unet



那stablediffusion的Unet有什么不一样呢，这里找到一张图，佩服这位小姐姐有耐心，借一下她的图



我解释一下和ResBlock模块和SpatialTransformer模块，输入为timestep_embedding，context 以及input就是三个输入分别是时间步数，文本向量，加噪图像，时间步数你可以理解为transformer里的位置编码，在自然语言处理中用来告诉模型一句话每个字的位置信息，不同的位置可能意思大不相同，而在这，加入时间步数信息，可以理解为告诉模型加入第几步噪声的时刻信息（当然这是我的理解）。

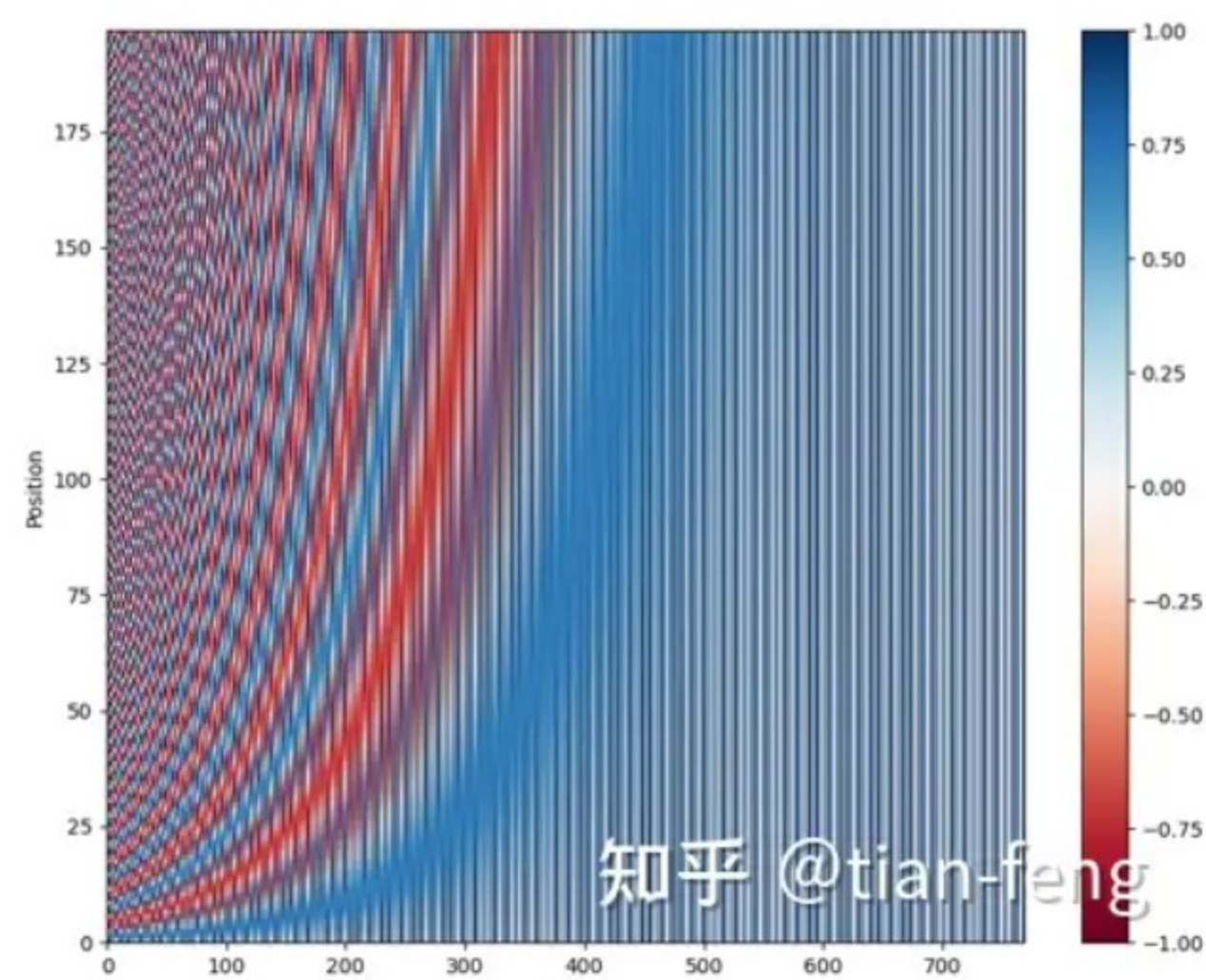
timestep_embedding采用正余弦编码



由于不同位置代表特征不同，加入位置编码是必须的。位置嵌入在维度上随着维度序号增大，周期变化会越来越慢，而产生一种包含位置信息的纹理。

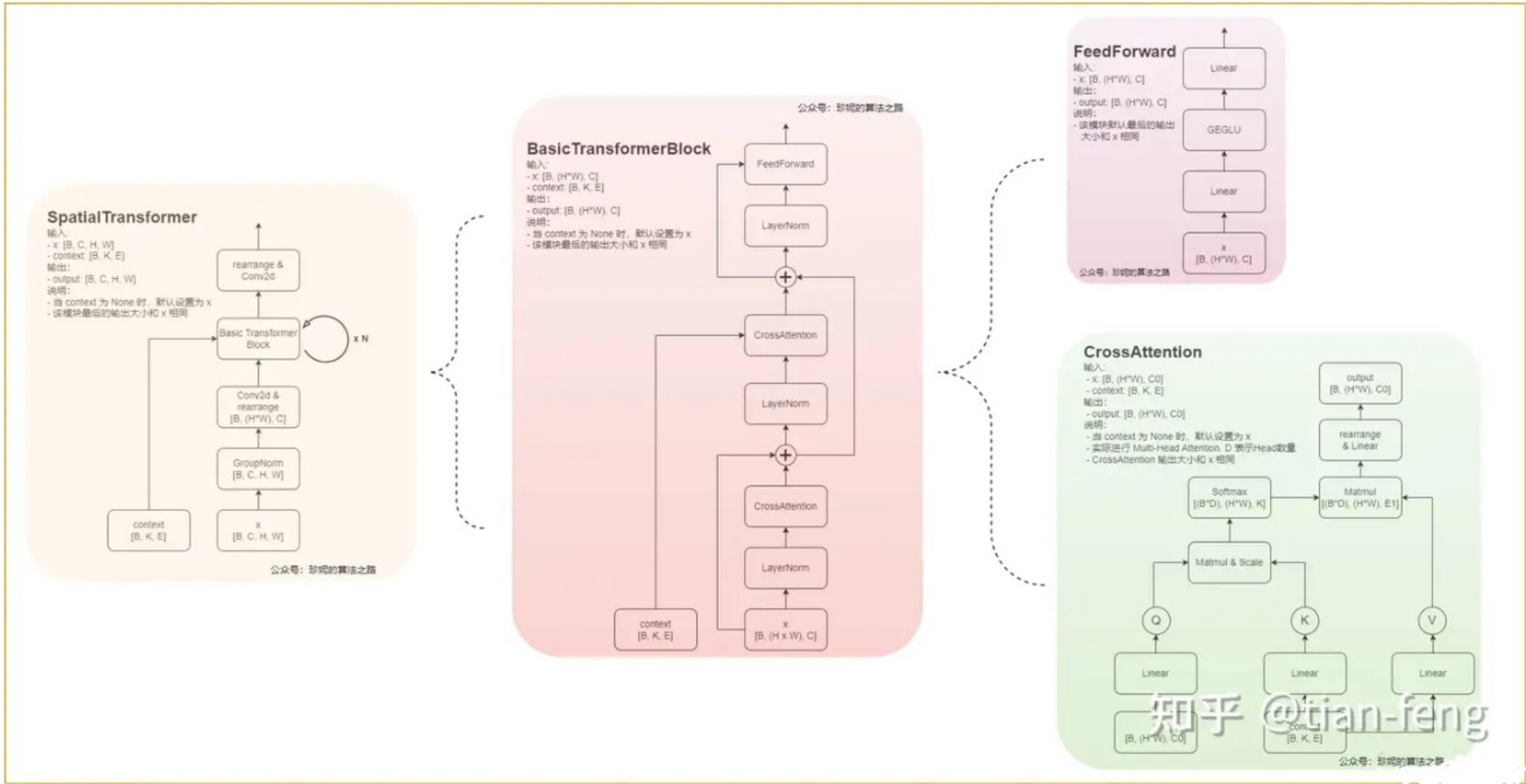
$$PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d_{\text{model}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d_{\text{model}}}\right)$$

pos代表竖轴，dmodel代表横轴



ResBlock模块输入为时间编码和卷积后图像输出，把它们相加，这就是它的作用，具体细节不说了，就是卷积，全连接，这些很简单。

SpatialTransformer模块输入为文本向量和上一步ResBlock的输出，



里面主要讲一下cross attention，其他都是一些维度的变换，卷积操作和各种归一化Group Norm，Layer norm，

利用cross attention将latent space（潜空间）的特征与另一模态序列（文本向量）的特征融合，并添加到diffusion model的逆向过程，通过Unet逆向预测每一步需要减少的噪音，通过GT噪音与预测噪音的损失函数计算梯度。

看右下角图，可以知道Q为latent space（潜空间）的特征，KV则都是文本向量连两个全连接得到，剩下就是正常的transformer操作了，QK相乘后，softmax得到一个分值，然后乘V，变换维度输出，你可以把transformer当做一个特征提取器，它可以把重要信息给我们显现出来（仅帮助理解），差不多就是这样了，之后操作都差不多，最后输出预测的噪声。

这里你肯定得熟悉transformer，知道什么是self attention，什么是cross attention不懂找篇文章看看，感觉不是可以简单解释清楚的。

完毕，拜拜，显示一些webui对比图

三、stable diffusion webui扩展

参数clip



clip skip为你选择跳过最后几层，适当clip skip可以保持不错效果，同时加快生成速度

知乎 @tian-feng

